scan/chain

An AgentMPI collective scaling analysis

Abstract

A scan is a prefix reduction, not a reduction to one place: rank i receives the fold over ranks $0 \ldots i$, so there are p different answers and every one of them must be produced. The chain is the obvious formulation — receive the prefix from $i-1$, fold your own contribution in, forward it to $i+1$ — and it is optimal on the two axes that count operations: $p-1$ messages and $p-1$ operator applications are both the floor, since every rank but the first must receive at least once and every contribution but the first must be folded at least once. Across 21 process counts from 2 to 32 the logged traffic is exactly $p-1$ at every size and the round count is exactly $p-1$ too, matching the closed form with no accounting gap and no dependence on whether p is a power of two. That equality of the two figures is the algorithm in one line: messages = rounds means every single message lies on the critical path, and the family is the sweep's only fully serial collective. The consequences are visible in the archive rather than merely derived. The $p = 32$ run takes 2.252 s — the longest wall time of any run in the collective sweep — to send 31 messages, while `scan/recursive_doubling` sends 129 in 0.245 s, and the trace shows the mechanism: the interval between successive sends grows from 10 ms early in the chain to about 240 ms late, which is the receive-side poll backoff saturating at its 250 ms ceiling. The second consequence is the one that matters for agent work: fold depth is also $p-1$, and it is distributed *unequally* — rank 0's answer is unfolded and rank 31's has been re-encoded 31 times. Under the cost model's default loss of $\delta = 0.05$ that is a surviving fidelity of 0.204 at the last rank against 0.774 under Hillis–Steele. The chain is the reference semantics and the only algorithm legal for a non-associative operator; it is not a scan a harness should choose for a semantic operator at any p above about four.

1 What this family is

The `scan` collective under the `chain` algorithm, measured at 21 process counts from 2 to 32. Every run is agent-free and deterministic, so the measurements are of the protocol itself.

2 What the analysis notices

- **[ok]** All 21 measured sizes logged exactly the number of messages the closed form predicts.
- **[note]** Wall times span 0.013–2.252 s with no agent in the loop, so these measure the fabric and the protocol rather than model latency.

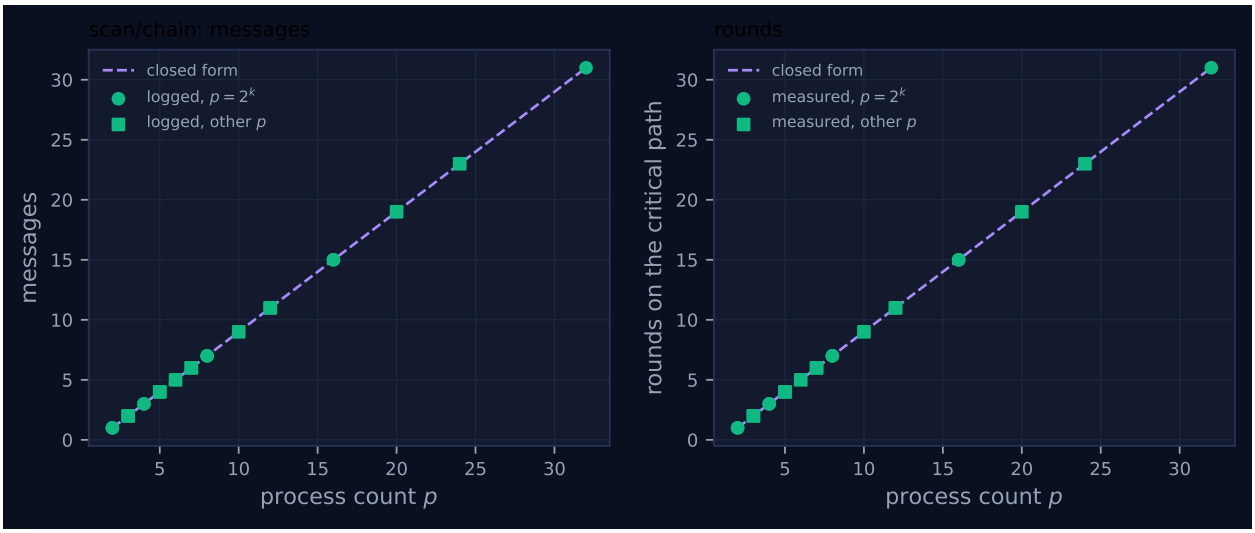

Figure 1: Measured message count and critical-path rounds against process count, with the closed-form prediction. Powers of two are drawn as circles and other sizes as squares, because algorithms take a different code path at sizes that are not powers of two. A red cross marks a size where the collective's own reported count disagrees with the traffic the fabric logged.

3 Every measured size

p	campaign	logged	pred.	self-rep.	rounds	pred.	wall (s)	model	acct.
2	`mb-free`	1	1	1	1	1	0.013	✓	✓
2	`mb-smoke`	1	1	1	1	1	0.014	✓	✓
3	`mb-free`	2	2	2	2	2	0.013	✓	✓
3	`mb-smoke`	2	2	2	2	2	0.026	✓	✓
4	`mb-free`	3	3	3	3	3	0.045	✓	✓
4	`mb-smoke`	3	3	3	3	3	0.043	✓	✓
5	`mb-free`	4	4	4	4	4	0.044	✓	✓
5	`mb-smoke`	4	4	4	4	4	0.067	✓	✓
6	`mb-free`	5	5	5	5	5	0.072	✓	✓
7	`mb-free`	6	6	6	6	6	0.081	✓	✓
7	`mb-smoke`	6	6	6	6	6	0.048	✓	✓
8	`mb-free`	7	7	7	7	7	0.103	✓	✓
8	`mb-smoke`	7	7	7	7	7	0.100	✓	✓
10	`mb-free`	9	9	9	9	9	0.131	✓	✓
12	`mb-free`	11	11	11	11	11	0.437	✓	✓
12	`mb-smoke`	11	11	11	11	11	0.140	✓	✓
16	`mb-free`	15	15	15	15	15	0.441	✓	✓
16	`mb-smoke`	15	15	15	15	15	0.996	✓	✓
20	`mb-free`	19	19	19	19	19	0.762	✓	✓
24	`mb-free`	23	23	23	23	23	0.771	✓	✓
32	`mb-free`	31	31	31	31	31	2.252	✓	✓

4 How the algorithm scales

4.1 Where the cost comes from

There is nothing to derive and that is the point of including it: the chain is what `Op.fold` does, spread over p processes. Rank $r > 0$ blocks on a receive from $r - 1$, computes `op.fn(incoming, payload)` — *incoming* on the left, so the evaluation order is exactly rank order — and if $r < p - 1$ forwards the result to $r + 1$. Rank 0 skips the receive; rank $p - 1$ skips the send.

So: $p - 1$ messages, one per non-final rank; $p - 1$ operator applications, one per non-initial rank; $p - 1$ rounds, because round r's send is round $r - 1$'s receive; and fold depth $p - 1$, because the value rank $p - 1$ holds has passed through every application. The `cost.FORMULAS` entry is $(p - 1,\ p - 1,\ (p - 1)n,\ p - 1)$, and every one of those four components is the same number.

Both message and application counts are minimal. Any correct scan must deliver information from rank 0 to rank $p - 1$, so at least $p - 1$ ranks receive at least one message; and the inclusive prefix at rank $p - 1$ is a fold over p contributions, which needs at least $p - 1$ binary applications. The chain attains both bounds simultaneously. Nothing else in the scan family does: `recursive_doubling` sends $p\lceil\log_2 p\rceil - (2^{\lceil\log_2 p\rceil} - 1)$ messages, which is 129 at $p = 32$ against 31.

4.2 What the figure shows

The two panels of Figure 1 are the same plot. Both are the straight line $p - 1$, both have the measured points exactly on it at all 21 rows, and both mix the two marker classes without any visible distinction. No other family in the sweep has identical panels, and the identity is the algorithm's defining property: a message count equal to a round count means the communication has no parallelism at all. Every one of the 31 messages at $p = 32$ is on the critical path, whereas in `allgather/bruck` at the same size 160 messages occupy 5 rounds, so 32 messages fly concurrently.

There are no red crosses. The self-report matches the logged traffic at all 21 rows, which is unsurprising for an algorithm with one `tr.sent()` call reachable by one code path.

The straight line also means this is the one family whose figure needs no caveat about interpolation. $p - 1$ is linear, so joining the measured grid with straight segments reproduces it exactly, and the marginal cost of one more rank really is the gradient: one message, one round, one level of fold depth, at every p.

4.3 What the trace adds: the wall time is a poll-backoff measurement

The wall-time column is the one place this family produces a number worth investigating rather than merely confirming. It runs from 0.013 s at $p = 2$ to 2.252 s at $p = 32$, which is the largest figure in the entire 450-run collective sweep, produced by the algorithm that sends the fewest messages at its size. The growth is also clearly superlinear: 0.103 s at $p = 8$, 0.441 s at $p = 16$, 2.252 s at $p = 32$ — roughly 4× per doubling for a 2× growth in messages.

The cause is visible in the log. Ordering the 31 `msg.send` events of `mb-free__coll-scan-chain-32` by time and differencing gives step intervals of 17.6 ms, 11.1 ms, 8.1 ms, 12.0 ms, 3.3 ms for the first few links, rising through 79.5 ms and 103.7 ms in the middle, and settling at 220.4 ms, 242.5 ms, 248.3 ms, 234.8 ms and 239.5 ms for the last handful. That ceiling is `MAX_POLL_INTERVAL` in `comm.py`: a blocked receive sleeps `POLL_INTERVAL` = 10 ms and multiplies the interval by 1.4 until it caps at

250 ms. A rank late in the chain has been waiting long enough to be sleeping at the cap when its predecessor's message finally lands, so it discovers the arrival up to a quarter of a second after it happened, and the chain pays that penalty once per link.

That is worth stating carefully, because it cuts both ways. It means the 2.252 s does not measure AgentMPI's protocol cost — the fabric work involved is 31 inserts and about 60 selects, tens of milliseconds' worth. It also means the wall-time comparison against `scan/recursive_doubling` (10.5× at $p = 32$) is an artefact-amplified version of a real structural difference (6.2× in rounds), and neither figure should be quoted as the other.

The collective's own `skew` metric confirms the serialisation independently and without reference to the backoff: at $p = 32$ the spread between the first and last rank to record the collective is 2.234 s against a collective span of 2.234 s. The two being equal to four decimal places says that no two ranks were ever inside the collective at different stages of progress — the span *is* the skew, which is what full serialisation looks like in a metric. Every other family has a skew well below its span.

The viewer screenshot for that run shows the same thing as a picture: a single clean diagonal staircase of green ticks descending one lane per step, 160 events, no work bars, and — because the steps widen — a diagonal that steepens towards the bottom of the frame.

5 Powers of two and everything else

There is no distinction to report, and unlike most families the reason is not that the remainder case is handled well but that the algorithm cannot tell the difference. There is no bit test anywhere in the `chain` branch: the neighbours are $r - 1$ and $r + 1$, the guards are $r > 0$ and $r < p - 1$, and the round count is $p - 1$. Powers of two are not special because nothing in the algorithm is indexed by $\log_2 p$.

Consequently this family is a control for the rest of the sweep. `allreduce/recursive_doubling` shows a self-report shortfall at every non-power-of-two process count and at none of the powers of two, and one then wants to know that the measurement apparatus is not itself sensitive to p's factorisation. Here is a family measured over the same 21 rows by the same tooling with the same two campaigns, in which the accounting column is ✓ at all 21 rows and the two marker classes are indistinguishable. The pattern there is in the algorithm, not in the harness.

The one p-dependent thing in the vicinity is algorithm *selection*, not execution. `scan`'s default is

```
"chain" if op.associativity is NONE else
("recursive_doubling" if p > 4 else "chain")
```

so the chain is what a caller gets below $p = 5$ or with a non-associative operator; every run in this family passed `algorithm="chain"` explicitly, so the sweep measures the algorithm and not the default.

6 When to choose this algorithm

6.1 The two cases where it is right

When the operator is declared non-associative, it is the only legal choice. Both `scan` and `reduce` raise `AmpiUsageError` if a tree algorithm is requested for an operator whose `associativity`

is `NONE`, on the same principle that stops MPI reordering a non-commutative operator, only stricter. The chain's left-operand discipline — `op.fn(incoming, payload)`, prefix on the left, own contribution on the right — is what makes it safe, and it means the chain evaluates precisely $((c_0 \circ c_1) \circ c_2) \circ \cdots$, the reference semantics that `agentmpi.ops.Op.fold` defines and that the other algorithms are tested against. This is not a performance argument and no crossover applies to it.

When p is small enough that serial depth is not yet expensive. The implementation's threshold is $p \leq 4$, and the cost model puts the crossover in the same place. Predicting both algorithms at a 1200-token payload with the repository's defaults ($\alpha = 12\,\text{s}$, $\beta^{-1} = 45$ tokens/s, $\delta = 0.05$):

	messages		modelled time (s)		modelled price (USD)		fidelity at $\delta = 0.05$	
p	chain	Hillis–St.	chain	Hillis–St.	chain	Hillis–St.	chain	Hillis–St.
3	2	3	77.6	77.6	0.0504	0.0576	0.9025	0.9025
4	3	5	116.5	77.6	0.0756	0.0900	0.8574	0.9025
8	7	17	271.7	116.5	0.1764	0.2484	0.6983	0.8574
16	15	49	582.3	155.3	0.3780	0.6228	0.4633	0.8145
32	31	129	1203.4	194.1	0.7812	1.4868	0.2039	0.7738

The two are indistinguishable at $p \leq 3$ and Hillis–Steele is ahead on time from $p = 4$ onwards, by $1.5\times$ at $p = 4$ and $6.2\times$ at $p = 32$. The chain is ahead on price at every size, and the ratio grows from $1.14\times$ at $p = 3$ to $1.90\times$ at $p = 32$. So the crossover is exactly where the code puts it: below five ranks the chain costs less and loses nothing worth having, and above five it loses a factor that grows as $p/\log_2 p$.

6.2 The case against it, which is about fidelity and is worse than the aggregate suggests

Fold depth $p - 1$ is the headline, and `cost.predict` reports the corresponding surviving fidelity for the whole collective as $0.95^{31} = 0.2039$ at $p = 32$ against Hillis–Steele's $0.95^5 = 0.7738$. That comparison understates the problem, because a scan has p answers and the aggregate figure is the worst of them.

The measured per-rank depths make the distribution explicit. In `mb-free__coll-scan-chain-32` the 32 `coll.scan` records report fold depths $0, 1, 2, \ldots, 31$ — one per rank, exactly its own index. Under $F(d) = (1 - \delta)^d$ that is a quality that decays geometrically along the rank axis: rank 0's prefix is its own contribution untouched, rank 7's has survived seven re-encodings at $F = 0.698$, rank 15's at $F = 0.463$, rank 31's at $F = 0.204$. Hillis–Steele's measured depths in the sibling family are $0, 1, 2, 2, 3, 3, 3, 3, 4, \ldots, 5$, that is $\lceil \log_2(r + 1) \rceil$, so its worst rank sits at $F = 0.774$ and the spread across ranks is small.

For the use case the `scan` docstring names — translating a novel where chapter i must be consistent with the names and register established in chapters $0 \ldots i - 1$ — this is the decisive property, and it is not a latency argument at all. Under a chain, the consistency information reaching the late chapters has been summarised and re-summarised once per intervening chapter, so the book gets *monotonically worse as it goes on*, which is exactly the failure a reader would notice. Under Hillis–Steele the last chapter's context has been through five folds rather than thirty-one. A harness that cares about output quality rather than throughput should still not use the chain.

6.3 What the chain does buy, stated precisely

$p - 1$ operator applications, against Hillis–Steele's much larger count. The message counts in the table above are also application counts for the chain (one fold per received message), but for Hillis–Steele they are a lower bound on it: reading the implementation, each received message triggers one `op.fn` for the inclusive accumulator and, from the second receive onwards, a second one for the exclusive accumulator, which an inclusive scan then discards. Summing the measured per-rank receive counts gives 227 applications at $p = 32$ against the chain's 31 — a factor of 7.3, not the 1.9 the price column shows, because `cost.predict` charges the operator-output term as $(p - 1) \cdot$ `op_cost_tokens` for every algorithm regardless of how many applications it actually performs.

That is the honest form of the trade-off, and it makes the chain's case stronger than the model does: the chain is the minimum-cost scan by a wide margin when each application is an agent invocation, and the question is only whether a harness can afford $p - 1$ sequential invocations of latency α. At $\alpha = 12\,\text{s}$ and $p = 32$ that is about 20 minutes of pure serial latency, and the answer is usually no.

7 Threats to this reading

No agents, so the fidelity argument — the main argument against this algorithm — is entirely modelled. All 21 members record `executors: {none: p}`, zero agent calls, zero tokens in and out, \$0.0000, `total_busy_s: 0` and `idle_fraction: 1.0`. The operator in every run is `SUM`, declared `Associativity.EXACT`, for which fold depth is irrelevant by construction: 31 additions lose nothing. The measured fold depths $0, 1, \ldots, 31$ are real and are the input the fidelity model wants, but $\delta = 0.05$ is `CostParams`'s default and not a measurement from this family; `bench_fidelity` and `benchmarks/bench_reduce.py` are where a measured δ would come from. Every fidelity number above should be read as "what the model says, given a per-application loss nobody measured here".

The wall-time figures measure a polling schedule and this family is the extreme case. 2.252 s for 31 messages is not a protocol cost; it is 31 links each paying up to `MAX_POLL_INTERVAL`. The step-interval sequence quoted in §1.3 is direct evidence, and the corroboration is that the intervals converge on 240 ms rather than growing without bound, which is what a capped exponential backoff predicts and what a genuine per-link cost would not. The one campaign-to-campaign check available is not reassuring about precision either: at $p = 16$ the two runs took 0.441 s and 0.996 s, a factor of 2.3 at fixed p. So the seconds column supports the qualitative claim (the chain's cost is dominated by its depth) and nothing quantitative.

The 2.252 s is the largest wall time in the collective sweep, and I have checked that against this sweep only. The claim is about the 450 collective runs. The archive's real-agent runs reach 7,200 s, so "longest run" would be wrong; "longest collective-sweep run" is what I mean and what the family metrics support.

The operator-application count for Hillis–Steele is read from the source, not logged. Nothing in the trace records an `op.fn` call — with an exact operator and no executor there is nothing to record — so the 227 figure comes from counting each rank's received messages in the log (129 in total at $p = 32$) and adding the second application per rank after the first, which the implementation performs unconditionally. The receive counts are measured; the mapping from receives to applications is a reading of six lines of `algorithms.scan`. A run with a real executor

would settle it by counting `agent.call` events, and none exists.

Every run in this family passed `algorithm="chain"` explicitly, so the family does not measure what a caller would get by default. At $p \geq 5$ with an exact operator the default is `recursive_doubling`, so no production harness would traverse this code path at the sizes where its cost is dramatic unless it asked to. The family is a measurement of the algorithm, not of the library's behaviour, and the $p = 32$ figures should not be read as something a user would encounter by accident.

Fold depth being exactly the rank index is a property of an unbroken chain, and every run here completed. All 21 members report `complete: true` with all p ranks participating. A chain is the most fragile possible topology — one dead rank stalls every rank above it, with no partial result and no absentee report, since `scan` takes a `timeout` and no `BarrierPolicy` — and nothing in this family exercises that. The fault-tolerance argument against the chain is therefore untested here, and it is probably a stronger argument than the latency one.